



الهيئة الاتحادية للهوية
والجنسية والجمارك وأمن المنافذ
FEDERAL AUTHORITY FOR IDENTITY,
CITIZENSHIP, CUSTOMS & PORT SECURITY

ID Card Toolkit Web Sample Build Instructions

Version 1.5

March 2026

CONFIDENTIAL

Table of Contents

Revision History	3
Abbreviations	4
1 Introduction	5
1.1 Purpose	5
2 Running the Pre-Built Sample with Web Start Agent	6
2.1 Prerequisites	6
2.2 Instructions	6
3 Running the Pre-Built Sample without Web Start Agent	7
3.1 Prerequisites	7
3.2 Instructions	7
4 Building the Web Sample with Web Start Agent	8
4.1 Prerequisites	8
4.2 Build Instructions	8
4.3 Running Instructions	8
5 Building the Web Sample without Web Start Agent	9
5.1 Prerequisites	9
5.2 Installing the Toolkit Agent Service	9
5.3 Build Instructions	9
5.4 Running Instructions	9
6 Web Console Sample Application	10
6.1 Overview	10
6.2 Prerequisites	10
6.3 Running the Console	10
6.3.1 Option 1: Via Web Server	10
6.3.2 Option 2: Direct File Access	10
6.4 Using the Console	10
7 Modern Web Sample	11
7.1 Overview	11
7.2 Prerequisites	11
7.3 Running the Modern Sample	11
7.3.1 Option 1: Via Web Server	11
7.3.2 Option 2: Direct File Access	11

Revision History

Version	Date	Description of Changes
1.0	04th Jun, 2018	Release Version 1.0.5
1.1	10th Jul, 2018	Release Version 1.0.7
1.2	15th Feb, 2019	Release Version 1.0.13
1.3	05th Mar, 2020	Release Version 1.4.5
1.4	20th Dec, 2023	Release Version 3.0.0
1.5	Mar 2026	Toolkit v3.0.5. Added Web Console sample application and modern web sample application documentation. Improved instructions.

Abbreviations

Abbreviation	Description
ICP	Federal Authority for Identity, Citizenship, Customs & Port Security
IDE	Integrated Development Environment
JNLP	Java Network Launch Protocol
SDK	Software Development Kit
SP	Service Provider
URL	Uniform Resource Locator

1. Introduction

The Federal Authority for Identity, Citizenship, Customs & Port Security (ICP) developed the ID Card Toolkit to address the requirements of service providers (SP) to integrate, into their business applications: Identification, Authentication, Digital Signature and Non-repudiation services, around the capabilities of the Emirates ID Card. The Toolkit is comprised of a number of Software Development Kits (SDK) supporting different programming languages and platforms.

1.1. Purpose

This document is a developer's reference guide to build and run the sample web applications provided with the ICP ID Card Toolkit Web SDK. The Toolkit ships two current web sample applications:

- **samples/web/console/** – lightweight browser test console with log-based output
- **samples/web/modern/** – modern WPF-style single-page application mirroring the .NET sample, with full card operations, PKI, biometrics, and digital signature pages

The legacy `ToolkitWebSample` web application is also covered below for existing integrations:

- **ToolkitWebSample** – full web application with Java Web Start agent (JNLP)
- **ToolkitWebSample (Agent mode)** – web application using the standalone Toolkit Agent service

2. Running the Pre-Built Sample with Web Start Agent

This section describes how to run the pre-built binary sample using the Java Web Start (JNLP) agent.

2.1. Prerequisites

- Java (JDK 1.7 or 1.8)
- ICP Web SDK jar file (IDCardToolkitService.jar)
- Java service for PIN encryption and XML signature verification (provided with the sample in samples/web/bin/ToolkitWebSample/WEB-INF/classes)
- Windows operating system
- Apache Tomcat 8+ server

2.2. Instructions

1. Download the Java Web Start jar file (IDCardToolkitService.jar) packaged for your organization.
2. Copy the jar file to samples/web/bin/ToolkitWebSample/.
3. Copy config_ag_ap to samples/web/bin/ToolkitWebSample/.
4. Copy SP-specific configuration files according to the environment parameter in config_ag_ap:

```
# If config_ag_ap contains: environment = dev
# Copy config files to: samples/web/bin/ToolkitWebSample/dev/

# If no environment is specified:
# Copy config files to: samples/web/bin/ToolkitWebSample/
```

5. Edit IDCardToolkitService.jnlp in the sample directory:

- Set the codebase attribute of the <jnlp> tag to your application URL.
- Set config_url in <arguments> to your application URL:

```
config_url = http://your-server:8080/ToolkitWebSample/
```

- (Optional in pre-production, mandatory in production) Set config_tls_cert to the Base64-encoded SSL certificate.
 - (Optional in pre-production, mandatory in production) Set config_tls_cert_chain to the Base64-encoded certificate chain.
6. Deploy the folder as an exploded WAR file to Apache Tomcat.
 7. Start Tomcat and access the application URL (e.g., http://your-server:8080/ToolkitWebSample).

3. Running the Pre-Built Sample without Web Start Agent

This section describes how to run the pre-built binary sample using the standalone Toolkit Agent service.

3.1. Prerequisites

- Java (JDK 1.7 or 1.8)
- Java service for PIN encryption and XML signature verification (provided with the sample)
- Windows operating system
- Apache Tomcat 8+ server
- Toolkit Agent service must be installed and running

3.2. Instructions

1. Copy SP-specific configuration files to `samples/web/bin/ToolkitWebSample/` (or the environment subdirectory as described above).
2. Deploy the folder as an exploded WAR file to Apache Tomcat.
3. Ensure the Toolkit Agent service is running.
4. Start Tomcat and access the application URL.

4. Building the Web Sample with Web Start Agent

4.1. Prerequisites

- Java (JDK 1.7 or 1.8)
- ICP Web SDK jar file (IDCardToolkitService.jar)
- Gradle (latest version)
- Windows operating system

4.2. Build Instructions

1. Copy IDCardToolkitService.jar to samples/web/src/main/webapp/.
2. Copy config_ag_ap to samples/web/src/main/webapp/.
3. Copy SP-specific configuration files according to the environment parameter:

```
# If environment = dev:  
# Copy to: samples/web/src/main/webapp/dev/  
  
# If no environment specified:  
# Copy to: samples/web/src/main/webapp/
```

4. Edit IDCardToolkitService.jnlp as described in Section 2 (set codebase, config_url, and optional TLS certificate parameters).
5. Build the WAR file:

```
cd samples/web  
gradle assemble
```

The WAR file is generated at samples/web/build/libs/ToolkitWebSample.war.

4.3. Running Instructions

1. Copy ToolkitWebSample.war to Tomcat's webapps directory.
2. Start the Tomcat server.
3. Open a browser and access the application URL (e.g., <http://your-server:8080/ToolkitWebSample>).
4. If the Toolkit Agent service is not running, a prompt will appear to download and launch it via JNLP.
5. The browser will download IDCardToolkitService.jnlp. Open it with Java Web Launcher.
6. Accept the security warning and click **Run**. This starts the Toolkit Agent service.
7. Refresh the page and click **Initialize** to connect to the Agent service. All Toolkit functionality will be enabled.

Note: If a Java security error occurs, add the application URL to the **Exception Site List** in the Java Control Panel's **Security** tab. Then restart the browser and try again.

5. Building the Web Sample without Web Start Agent

5.1. Prerequisites

- Java (JDK 1.7 or 1.8)
- Gradle (latest version)
- Windows operating system
- Toolkit Agent service must be installed and running

5.2. Installing the Toolkit Agent Service

1. Copy the Agent Service binary `EIDAToolkitService.exe` from `bin/agent/<platform>` of the Toolkit distribution to the desired installation directory.
2. Place `config_ag_ap` in the Agent directory.
3. Place SP-specific configuration files in the Agent directory (or the `environment` subdirectory).
4. Place all dependent libraries (Toolkit library and plugins) in the Agent directory.
5. Install the service from a command prompt:

```
EIDAToolkitService.exe -install
```

5.3. Build Instructions

1. Copy `config_ag_ap` to `samples/web/src/main/webapp/`.
2. Copy SP-specific configuration files according to the `environment` parameter.
3. Build the WAR file:

```
cd samples/web  
gradle assemble
```

The WAR file is generated at `samples/web/build/libs/ToolkitWebSample.war`.

5.4. Running Instructions

1. Copy `ToolkitWebSample.war` to Tomcat's `webapps` directory.
2. Start the Tomcat server.
3. Ensure the Toolkit Agent service is running.
4. Open a browser and access the application URL.
5. Click **Initialize** to connect to the running Agent service. All Toolkit functionality will be enabled.

6. Web Console Sample Application

The Toolkit distribution includes a standalone HTML-based sample application demonstrating all SDK operations in a browser-based interface.

6.1. Overview

The Web Console is a single-page HTML application (`samples/web/console/index.html`) that provides a browser-based interface to all Toolkit JavaScript SDK functions. It features:

- **Sidebar navigation** with categorized operation buttons (Connection, Card Info, Public Data, PKI & Signing, Biometric, PIN Management, NFC, Digital Docs, AdES Signing)
- **Dark-themed output console** with color-coded log entries (info, success, error, data)
- **Status bar** showing connection state and card information
- **No build step required** – runs directly from the file system or any web server

6.2. Prerequisites

- Toolkit Agent service or Web Start agent running
- Modern web browser (Chrome, Firefox, Edge)
- ICP Toolkit configuration files in the Agent's working directory

6.3. Running the Console

6.3.1. Option 1: Via Web Server

Copy the `samples/web/console/` folder to a web server (e.g., Tomcat) and access `index.html` through the browser.

6.3.2. Option 2: Direct File Access

Open `samples/web/console/index.html` directly in a browser.

Note: When opening from the file system, some browsers may block WebSocket connections. Use a web server for full functionality.

6.4. Using the Console

1. Click **Initialize** to connect to the Toolkit Agent service.
2. Use **List Readers** to discover connected smartcard readers.
3. Click **Connect** to establish a connection to the first reader with a card.
4. Use the operation buttons in the sidebar to test individual Toolkit functions.
5. All responses and errors are displayed in the output console with timestamps.
6. Click **Cleanup** when finished to release Toolkit resources.

7. Modern Web Sample

The modern web sample (`samples/web/modern/`) is a single-page web application that mirrors the modern .NET MVVM sample. It provides a richer, navigable UI with dedicated pages for card operations, public data, PKI and signing, biometrics, PIN management, NFC, and digital signature (AdES).

7.1. Overview

The modern sample is organized into pages equivalent to the modern .NET sample:

- **Card Operations** – CSN, card status, device ID, license expiry, register device
- **Public Data** – non-modifiable data, modifiable data, home/work address, photo, signature image
- **PKI & Signing** – authenticate PKI, sign/verify data with auth and sign certificates
- **Biometric** – biometric-on-server, card + biometric authentication
- **PIN Management** – reset PIN, unblock PIN, PKI certificates
- **NFC** – set/clear NFC auth parameters, read public data via NFC
- **AdES Signing** – PAdES, XAdES, and CAdES signature creation and verification

7.2. Prerequisites

- Toolkit Agent service or Web Start agent running
- Modern web browser (Chrome, Firefox, Edge)
- ICP Toolkit configuration files in the Agent's working directory

7.3. Running the Modern Sample

7.3.1. Option 1: Via Web Server

Copy the `samples/web/modern/` folder to a web server (e.g., Tomcat) and access `index.html` through the browser.

7.3.2. Option 2: Direct File Access

Open `samples/web/modern/index.html` directly in a browser.

Note: When opening from the file system, some browsers may block WebSocket connections. Use a web server for full functionality.

Note: The `holderSignatureImage` field returned by the Validation Gateway is in TIFF format (CCITT compressed). Web browsers cannot render TIFF images natively. Applications requiring display of the holder signature image should convert the TIFF data to a browser-compatible format (PNG or JPEG) before rendering. The raw TIFF data is still included in the XML response and accessible to the application.